

Live Check-in Feature — Walkthrough

What Was Built

A multi-peer mesh WebRTC video/audio "Live Check-in" for TaskTrove communities, with all signaling done via Firestore (no backend server needed).

Files Changed

File	Action	Description
liveCheckinWebrtc.ts	NEW	LiveCheckinRoomManager class — full mesh WebRTC with Firestore signaling
LiveCheckinDialog.tsx	REWRITE	Multi-peer video dialog with pre-join screen, responsive grid, controls
page.tsx	MODIFY	Pass <code>communityId={id}</code> to LiveCheckinDialog
firestore.rules	MODIFY	Added <code>liveRooms/{roomId}/participants + signals</code> security rules

Architecture Overview

```
graph LR
    A["User A Browser"] -- "RTCPeerConnection" --> B["User B Browser"]
    A -- "RTCPeerConnection" --> C["User C Browser"]
    B -- "RTCPeerConnection" --> C
    A -- "Firestore writes" --> F["Firestore Signaling"]
    B -- "Firestore writes" --> F
    C -- "Firestore writes" --> F
    F -- "onSnapshot" --> A
    F -- "onSnapshot" --> B
    F -- "onSnapshot" --> C
```

- **Mesh topology:** N-1 connections per user, practical limit ~6 users
- **Deterministic offer direction:** lower UID sends the offer (prevents duplicate offers)
- **Heartbeat:** `lastActive` updated every 30s, stale peers (>60s) are cleaned up
- **ICE:** Google free STUN servers (no TURN — may fail behind restrictive NATs)

Firestore Data Structure (new)

```
communities/{communityId}/liveRooms/{roomId}/
├ participants/{userId}    → { displayName, avatarUrl, joinedAt,
lastActive }
└ signals/{autoId}        → { type, from, to, payload, createdAt
}
```

Verification

Typecheck

`npm run typecheck` passes with **0 new errors**. The 7 errors reported are all pre-existing (Next.js 15 params typing and AppLayout issues).

Manual Testing

To test locally:

1. Run `npm run dev` (starts on port 9002)
2. Open two browser windows, each logged in as a different user
3. Navigate to the same community → **Live Check-ins** tab
4. Click **Start a Check-in** in both → grant camera/mic
5. Confirm both see each other's video tiles
6. Test mute, video-off, and leave buttons

[!NOTE] For testing across networks, a TURN server would be needed. localhost testing works peer-to-peer without one.